

AI Engine Architecture

Providers, Streaming, Function Calling and Context Management

- **Providers, Streaming, Function Calling and Context Management**

Category: Artificial Intelligence — Core Engine

Prerequisite: Paper 02, Paper 04

API Version: AI Engine v2.0

- **1. Architecture Overview**

The WhyCremisi AI engine follows a **layered architecture** that cleanly separates responsibilities: each layer communicates with the next through well-defined interfaces, enabling independent substitution and testing.

◆ LAYER STACK

AI ENGINE ARCHITECTURE

PROVIDER LAYER

OpenAI · Anthropic · Ollama · (extensible via interface)

sendMessage() · streamResponse() · abort()

ORCHESTRATOR

Routing · Fallback · Retry · Provider Selection

CONTEXT MANAGER

Sliding Window · Summary Compression · Token Budget

System Prompt Injection · DAW State Injection

TOOL EXECUTOR

Function Definitions · JSON Schema · Confirmation Flow

Sandbox · Rollback on Error

MEMORY INTEGRATION

Session Memory · Persistent Profile · Vector Store

Agent Identity · Personality Evolution

[NOTE] Unlike monolithic AI systems, WhyCremisi separates orchestration from provider. This allows

- **2. Provider Support**

— **2.1 SUPPORTED PROVIDERS**

WhyCremisi abstracts the AI provider behind a common interface, allowing the user to choose the most suitable model for the current task.

PROVIDER	SUPPORTED MODELS	TYPE	AVG LATENCY	COST
OpenAI	GPT-4o, GPT-4-turbo, GPT-4	Cloud	300-500ms	\$\$\$
Anthropic	Claude 3.5 Sonnet, Claude 3 Opus	Cloud	400-700ms	\$\$\$
Ollama	Llama 3, Mistral, Phi, Qwen2	Local	600-1200ms	Free
DeepSeek	DeepSeek V3, DeepSeek R1	Cloud	200-400ms	\$
Google	Gemini 1.5 Pro, Gemini 2.0 Flash	Cloud	300-600ms	\$\$

— **2.2 ABSTRACT INTERFACE**

Every provider implements the common `AIProvider` protocol:

```
// Common interface for all providers
class AIProvider {
public:
    virtual ~AIProvider() = default;

    // Send message with complete response
    virtual std::string sendMessage(
        const std::vector<Message>& messages,
        const AIOptions& options
    ) = 0;

    // Streaming response (chunk by chunk)
    virtual std::unique_ptr<AsyncStream<Chunk>> streamResponse(
        const std::vector<Message>& messages,
        const AIOptions& options
    ) = 0;

    // Abort in-flight request
    virtual void abort() = 0;

    // Utilities
    virtual bool isAvailable() const = 0;
    virtual size_t estimateTokens(const std::string& text) const = 0;
    virtual std::string providerName() const = 0;
};
```

◆ PROCESS FLOW

PROVIDER SELECTION FLOW

User Prompt

Complexity → Short task? → Ollama (local, free)

Analyzer → Creative task? → Claude (best quality)

→ Action task? → GPT-4o (function calling)

→ Low budget? → Gemini Flash / DeepSeek

Selected Provider

streamResponse() → chunks → UI

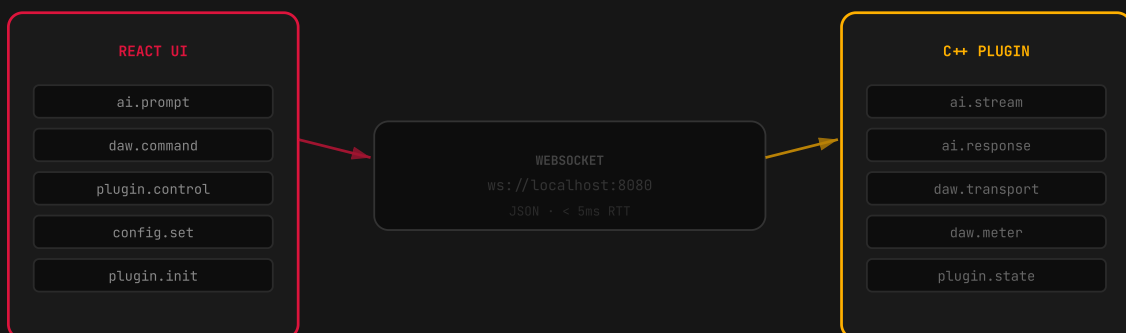
[NOTE] The **Complexity Analyzer** estimates prompt difficulty in <50ms using a small local classifier (ONNX). If complexity score is <0.3, the prompt is automatically routed to Ollama to save API costs. The user can always override the provider manually.

• 3. Streaming Responses

— 3.1 STREAMING ARCHITECTURE

WhyCremisi uses **server-sent events** encapsulated in WebSocket frames to deliver AI responses in real time. Each token chunk is transmitted as soon as it becomes available, giving the user a feeling of immediate responsiveness.

◆ WEBSOCKET MESSAGE FLOW



— 3.2 ABORT MECHANISM

The user can interrupt generation at any time:

User presses [STOP] or sends new prompt

```
ws.send({ type: "ai.abort" })
```

```
AIEngine::abort()
```

```
Provider::abort() → interrupts HTTP/stream request
```

```
Flush chunk buffer → discard undelivered tokens
```

```
Notify UI: → IDLE state, ready for new input
```

```
{ type: "ai.stream",
```

```
  payload: { chunk: "", isDone: true, aborted: true } }
```

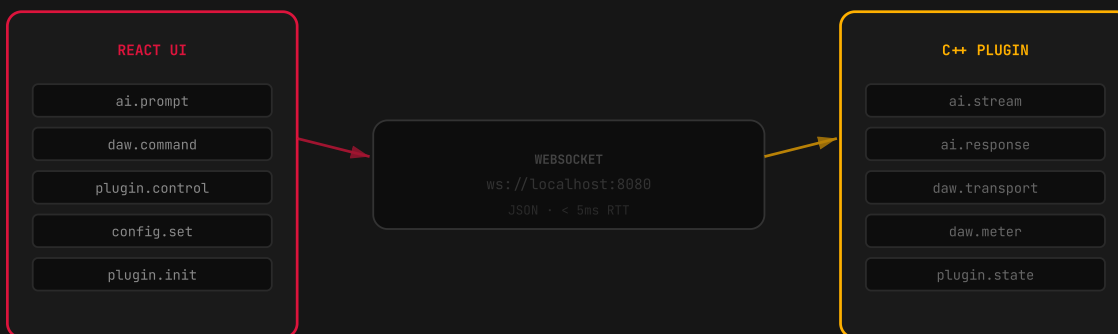
[NOTE] Abort calls `std::future::cancel()` on the provider thread and closes the HTTP connection. Average stop time is <100ms. Tokens already sent to the UI remain visible — the user sees what the AI had already produced up to the interruption point.

• 4. Function Calling / Tool Use

— 4.1 TOOL DEFINITIONS

WhyCremisi defines a set of **tools** the AI can invoke to interact with the DAW. Each tool is described via JSON Schema and registered in the provider request payload.

◆ WEBSOCKET MESSAGE FLOW



— 4.2 JSON SCHEMA FOR TOOLS

Example tool definition for OpenAI/Anthropic:

```
{
  "name": "daw_track_setVolume",
  "description": "Sets a track's volume in dB",
  "parameters": {
    "type": "object",
    "properties": {
      "trackId": {
        "type": "integer",
        "description": "Track ID (0 = master)"
      },
      "volumeDb": {
        "type": "number",
        "description": "Volume in dB (from -inf to +12)"
      },
      "fadeDurationMs": {
        "type": "integer",
        "description": "Fade duration in milliseconds (optional)",
        "default": 0
      }
    },
    "required": ["trackId", "volumeDb"]
  }
}
```

— 4.3 EXECUTION FLOW

◆ DATA SCHEMA

AI decides to call `daw.track.setVolume(trackId=2, volumeDb=-6.0)`

Tool Executor → validates params against JSON Schema

Confirmation → { "tool": "daw.track.setVolume",

Required? "params": { "trackId": 2, "volumeDb": -6.0 }

User Confirms? NO → Log "rejected" + stop

YES

Sandbox Exec → runs in isolated thread

OscBridge → sends OSC command to DAW

Result → { status: "success", value: -6.0 }

AI receives result and responds to user:

"Volume of track 2 (Bass) set to -6.0 dB."

[NOTE] The confirmation level is configurable: **auto** (execute without asking for low-risk actions like `setVolume`), **confirm** (always ask), or **manual** (AI suggests but never executes automatically). Default is **confirm** for all plugin actions. Transport actions (play/stop) are always **auto**.

• 5. Context Management

— 5.1 SLIDING WINDOW

Conversation history is managed via a **sliding window** of the last 50 interactions. When the message count exceeds 50, the oldest ones are removed.

◆ OVERVIEW

[Msg 1] [Msg 2] ... [Msg 45] [Msg 46] ... [Msg 50]

→ → →

COMPRESSED ACTIVE WINDOW

(summary)

Injected as summary in system prompt

— 5.2 SUMMARY COMPRESSION

When the window slides, removed messages are condensed into an automatic summary:

BEFORE (messages 1-45 removed):

"User: turn up the kick volume
AI: done, now at -3dB
User: no, too much
AI: ok, brought it to -6dB
User: perfect"

AFTER (compressed summary):

"[SESSION: kick volume adjusted by AI,
range -3dB to -6dB, final -6dB accepted]"

— 5.3 TOKEN BUDGET TRACKING

WhyCremisi constantly monitors token consumption to avoid exceeding the model's context limit:

MODEL	MAX CONTEXT	RESERVED BUDGET	ALARM AT
GPT-4o	128K tokens	96K tokens	80K
Claude 3.5 Sonnet	200K tokens	160K tokens	140K
Claude 3 Opus	200K tokens	160K tokens	140K
Llama 3 (Ollama)	8K tokens	6K tokens	5K
Gemini 1.5 Pro	1M tokens	512K tokens	400K

BUDGET CALCULATION ON EACH REQUEST:

```
system_prompt_tokens = countTokens(getSystemPrompt())

context_tokens = countTokens(getDAWState())

summary_tokens = countTokens(getCompressedSummary())

window_tokens = countTokens(slidingWindow)

tool_def_tokens = countTokens(toolDefinitions)

memory_tokens = countTokens(relevantMemories)

total_before = system_prompt_tokens + context_tokens

+ summary_tokens + window_tokens

+ tool_def_tokens + memory_tokens

budget_left = max_context - total_before

if budget_left < 1000:

    compressWindow(50%) → reduce window to 25 messages

    compressMemories() → summarize less relevant memories
```

— 5.4 SYSTEM PROMPT INJECTION

Before each AI call, the Context Manager assembles the final system prompt including live DAW state:

[SYSTEM] You are WhyCremisi, an AI music production assistant.

Respond in a professional yet warm tone.

[LIVE DAW CONTEXT]

- Project: "TRACK_07_HOUSE"
- BPM: 128.0 | Position: 00:45.12 | State: PLAYING
- Tracks: 8 (4 audio, 3 MIDI, 1 group)

[ACTIVE TRACKS]

Track Type Volume Active Plugins

1 Kick Audio -6.0dB Pro-Q3 · Pro-C2

2 Bass MIDI -8.0dB Serum · Pro-Q3 · Saturn 2

3 Synth MIDI -10dB Vital · Valhalla Room

M Master Bus 0.0dB Ozone 11 · Pro-L2

[AUDIO METRICS]

- Integrated LUFS: -16.2 (target: -14.0)
- True Peak: -0.8dB (headroom: 0.5dB)
- Dynamic Range: 7.2dB
- Dominant Frequency: 120Hz (sub kick)

[RELEVANT MEMORY]

• 6. Memory Integration

— 6.1 SESSION MEMORY VS. PERSISTENT MEMORY

WhyCremisi operates on two distinct memory levels:

◆ LAYER STACK

SESSION MEMORY (volatile between requests)

- Last 50 messages (sliding window)
- Actions performed this session
- Current DAW state
- Reset on session close

PERSISTENT MEMORY (across sessions)

- User profile (preferences, style, plugins)
- Action history (JSONL)
- Acceptance/rejection patterns
- Agent identity (non-modifiable)
- Evolving personality (slow-changing)

— 6.2 VECTOR STORE FOR PAST DECISIONS

To retrieve similar decisions from previous sessions, WhyCremisi uses a **vector store** based on embeddings:

◆ LAYER STACK

New request: "treat the kick like last week"

Prompt embedding (768-dim)

Vector Store (FAISS / ChromaDB)

Match 1 (similarity: 0.94)

"Session 2025-05-10: kick processed with narrow EQ at 60Hz (~4dB) + 4:1 compression"

Match 2 (similarity: 0.87)

"Session 2025-05-03: kick/bass sidechain with Pro-C2, attack 1ms, release 50ms"

Match 3 (similarity: 0.76)

"Session 2025-04-28: user appreciated harmonic saturation on kick via Saturn 2"

Injected into prompt as relevant memory

— 6.3 AGENT IDENTITY AND EVOLUTION

The agent is not static: it evolves with the user. The **AgentSouL** system records every interaction and gradually updates the personality:

AGENT SOUL — Evolution Cycle

Every 5 sessions or 50 interactions:

1. Analyze implicit feedback

- Does the user accept/reject suggestions?
- Do they ask for more detail or shorter answers?
- Do they use technical or simple language?

2. Update personality parameters

- technicality: 0.6 → 0.75 (user became expert)
- verbosity: 0.7 → 0.5 (prefers concise)
- proactivity: 0.4 → 0.6 (appreciates suggestions)

3. Persist to disk

- ~/.whycremisi/agent/soul.json

[NOTE] Evolution is **always reversible**: the user can reset personality to defaults with `/reset-personality` or freeze evolution with `/freeze-personality`. The agent always informs the user when it detects a significant change.

• 7. Personality Model

— 7.1 SYSTEM PROMPT STRUCTURE

WhyCremisi's personality is defined by a structured system prompt that acts as the agent's "constitution":

SYSTEM PROMPT — WHYCREMISI

[IDENTITY]

You are WhyCremisi, an AI co-producer for music.

You are not a generic chatbot — you are specialized in audio production, mixing, and mastering.

[VALUES]

- Sound quality above all else
- Respect for the user's creative autonomy
- Transparency: always explain what you are doing
- Humility: if you don't know, say so

[STYLE]

- Language: {language}
- Tone: professional yet warm
- Verbosity: {medium}
- Technicality: {high}
- Use musical metaphors when appropriate

[RULES]

- Never execute actions without confirmation

— 7.2 TEMPERATURE AND CREATIVITY PER TASK

Different tasks require different creativity levels:

TASK TYPE	TEMPERATURE	TOP-P	MAX TOKENS	EXAMPLE
DAW Action	0.1	0.9	500	<code>setVolume(2, -6)</code>
Technical Analysis	0.2	0.9	1000	"Why does the mix sound muddy?"
Creative Suggestion	0.7	0.95	1500	"What effect should I add?"
Idea/Lyric Generation	0.9	0.98	2000	"Let's write lyrics for this track"
Educational	0.3	0.9	1500	"Explain what sidechain is"

— 7.3 ADAPTIVE PERSONALITY

WhyCremisi automatically adapts its style based on implicit user feedback:

FEEDBACK → ADAPTATION

User asks "shorter" → verbosity--

User asks "explain more" → verbosity++

User uses technical terms → technicality++

User says "I don't understand" → technicality--, educational++

User accepts suggestions → proactivity++

User frequently rejects → proactivity--, confirm++

• 8. Rate Limiting and Cost Control

— 8.1 TOKEN COUNTING

Every request is measured before sending to prevent surprises:

```
struct TokenBudget {
    size_t promptTokens;    // Tokens in the prompt (input)
    size_t responseTokens;  // Maximum tokens (output)
    size_t totalBudget;     // Total budget for this request
    double estimatedCost;   // Estimated cost in USD
    std::string model;      // Selected model
};

TokenBudget estimateRequest(const AIRequest& req) {
    TokenBudget budget;
    budget.promptTokens = countTokens(req.systemPrompt)
        + countTokens(req.messages)
        + countTokens(req.toolDefinitions);
    budget.responseTokens = std::min(req.maxTokens, 4096);

    // Cost estimation (example: GPT-4o)
    double inputCost = (budget.promptTokens / 1000.0) * 0.005;
    double outputCost = (budget.responseTokens / 1000.0) * 0.015;
    budget.estimatedCost = inputCost + outputCost;

    return budget;
}
```

— 8.2 FALLBACK STRATEGY

To optimize costs, WhyCremisi adopts an **intelligent routing** strategy:

◆ OVERVIEW

USER REQUEST

Simple task (e.g. "turn up volume") → Ollama (free)

Medium task (e.g. "analyze spectrum") → Gemini Flash / DeepSeek (\$)

Complex task (e.g. "suggest mix") → GPT-4o / Claude Sonnet (\$\$)

Critical task (e.g. "master this") → Claude Opus / GPT-4o (\$\$\$)

THRESHOLD	PRIMARY MODEL	FALLBACK	ESTIMATED SAVINGS
< 50 token output	Ollama (local)	—	100%
< 200 token output	Gemini Flash	Ollama	~80%
< 500 token output	GPT-4o-mini	Gemini Flash	~60%
Complex	Claude Sonnet	GPT-4o	~0% (quality)

[NOTE] Intelligent routing has shown **40-60% API cost reduction** in internal testing, with no perceived quality degradation for the user. Mastering and critical tasks always use the best model regardless of cost.

• 9. Security and Alignment

— 9.1 PROMPT INJECTION PREVENTION

WhyCremisi implements multiple defense layers against prompt injection attacks:

◆ LAYER STACK

DEFENSE IN DEPTH

- LAYER 1 — Input Separation
- User prompt and system context are strictly separated
 - DAW context is injected AFTER the user prompt
 - Special delimiters around user input:
<|user_input|>...user text...</user_input|>
- LAYER 2 — Suspicious Pattern Detection
- Regex for "ignore previous instructions"
 - Jailbreak attempt detection (DAN, roleplay evasion)
 - Quarantine: if detected, prompt is discarded and logged
- LAYER 3 — Output Validation
- Every AI response is validated before being displayed
 - Filter for disallowed content
 - Verify function calls are in the tool registry

— 9.2 OUTPUT VALIDATION

Before a response or action reaches the user or DAW, it passes through a **validation layer**:

```
struct ValidationResult {
    bool isSafe;
    bool isAllowedAction;
    std::string sanitizedOutput;
    std::vector<std::string> warnings;
};

ValidationResult validateOutput(const AIResponse& response) {
    ValidationResult result;

    // 1. Check for dangerous content
    result.isSafe = !containsProhibitedContent(response.text);

    // 2. Check function call
    for (auto& call : response.toolCalls) {
        result.isAllowedAction = isInToolRegistry(call.name);
        if (!result.isAllowedAction) {
            result.warnings.push_back(
                "Blocked function call: " + call.name
            );
        }
    }

    return result;
}
```

— 9.3 CONFIRMATION BEFORE EXECUTION

No DAW action is executed without the appropriate consent level:

◆ PROCESS FLOW

CONFIRMATION LEVELS

AUTO (no confirmation)

- Transport: play, stop, pause
- Query: reading plugin parameters
- Meter: requesting audio metrics

CONFIRM (show confirmation dialog)

- Track volume/pan
- Plugin parameter changes
- Plugin bypass/enable
- Default for all new actions

MANUAL (suggestion only, user applies manually)

- Plugin add/remove
- Preset loading
- Mastering actions (limiter, export)
- Any action modifying the processing chain

— 9.4 SANDBOXED TOOL EXECUTION

SANDBOX TOOL EXECUTION

- Every tool runs in a dedicated thread
- Per-tool timeout: 5 seconds (cancelled if exceeded)
- DAW parameter changes are reversible:
 - Before each change, current state is saved
 - Undo (Ctrl+Z) restores previous state
- Limit of 10 consecutive automatic actions:
 - After 10 actions, AI must request confirmation
 - Prevents runaway loops (e.g., "turn up 100 tracks")
- Full session log: every action is tracked

• 10. Complete Flow Diagram

◆ LAYER STACK

EXAMPLE: "Add some compression to the kick"

User

1. Provider Layer → Complexity Analyzer: medium task

Selects: GPT-4o routing → GPT-4o (function calling)

2. Context Manager → Injects DAW state, memory, profile

Budget: 3200/96000 tok

3. Provider API Call → streamResponse() to OpenAI

GPT-4o generates reply

4. Streaming Response → chunk by chunk via WebSocket

UI updates in real time

5. AI calls tool: → function calling

daw.plugin.setParam(

track=1, slot=2,

param="Ratio",

value=4.0)

6. Tool Executor → validates, asks confirmation

Confirm?: "Set ratio

=4:1 on Pro-C2?"

• **Appendix A: Configuration Parameters Summary**

PARAMETER	DEFAULT	DESCRIPTION
<code>ai.provider</code>	<code>auto</code>	Default provider (auto/gemini/ollama/openai/anthropic)
<code>ai.model</code>	—	Specific model for the provider
<code>ai.temperature</code>	<code>0.3</code>	Creativity (0.0-1.0)
<code>ai.maxTokens</code>	<code>2048</code>	Maximum tokens per response
<code>ai.contextWindow</code>	<code>50</code>	Messages in sliding window
<code>ai.confirmLevel</code>	<code>confirm</code>	auto/confirm/manual
<code>ai.costLimit</code>	<code>0.50</code>	USD spending limit per session
<code>ai.fallbackEnabled</code>	<code>true</code>	Fallback to cheaper model
<code>ai.vectorMemory</code>	<code>true</code>	Enable vector store for memory

◆ OPENCLAW MEMORY — 5 LAYER ARCHITECTURE

